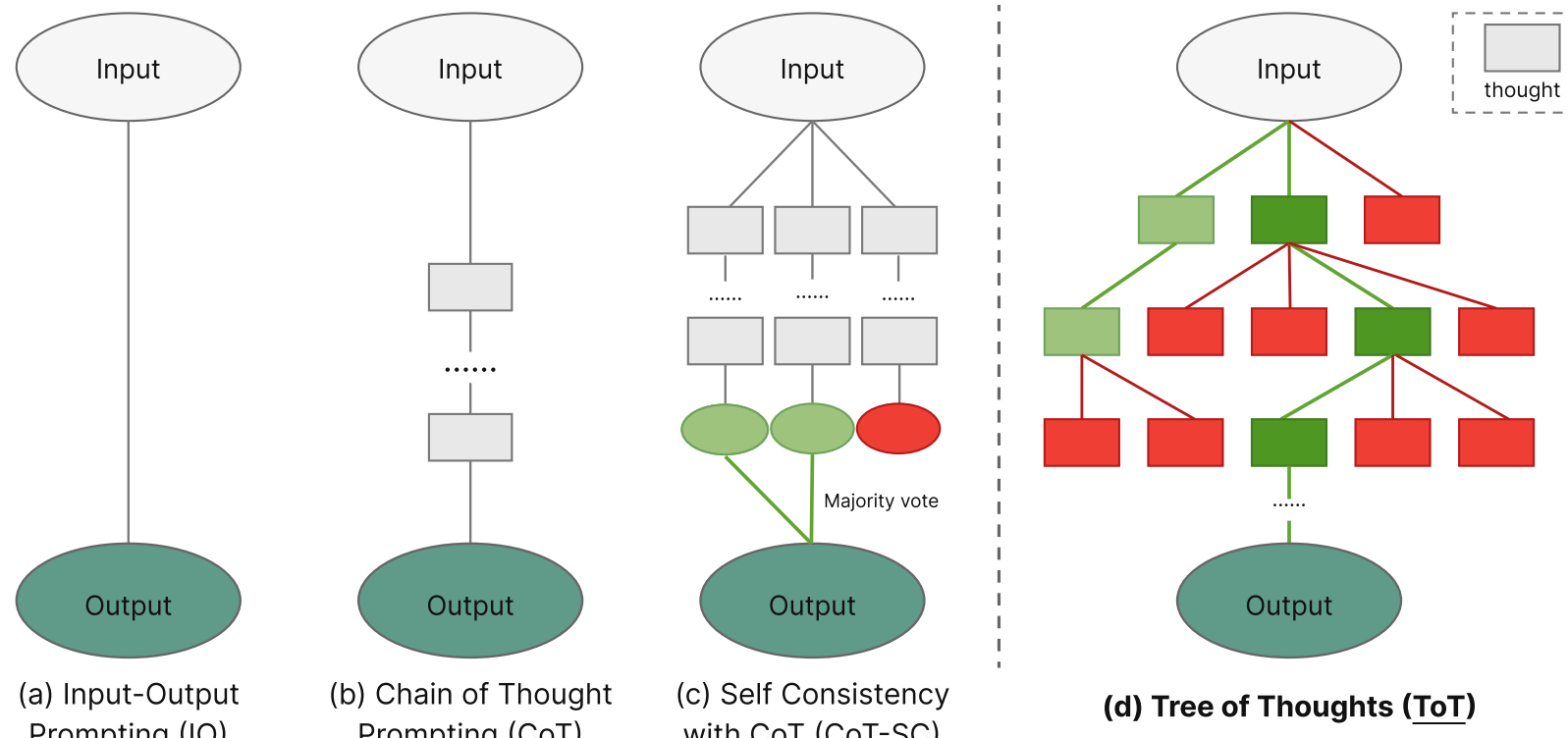


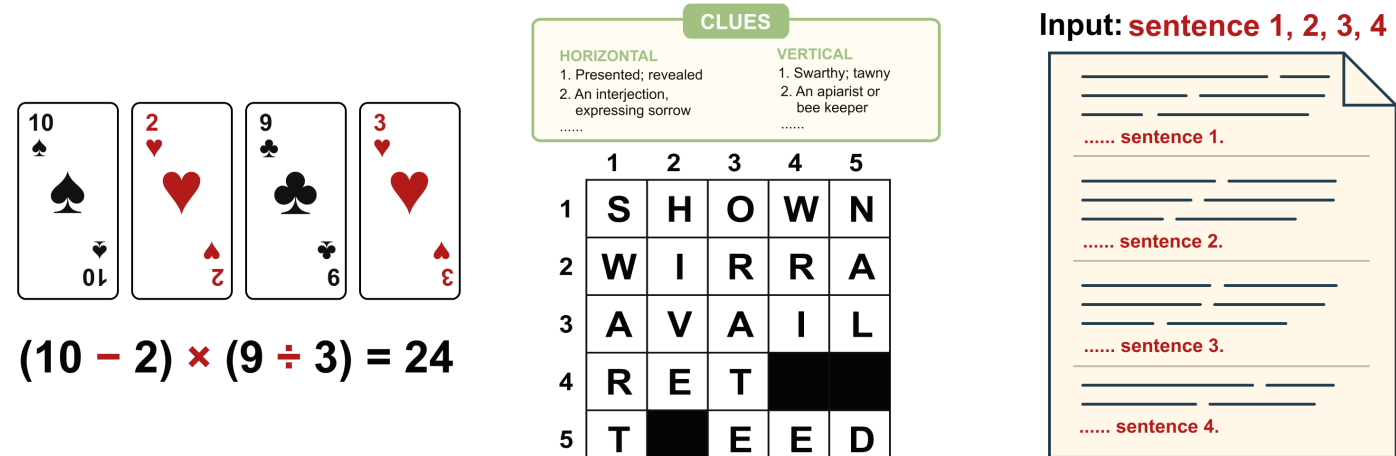
Improving Tree of Thoughts with Batching and Refinement

Minghan Gao (mg2328) Warren Hua (wsh48) Zhichun Zhang (zz547) Yihan Zhao (yz2788)

Why Tree of Thoughts?



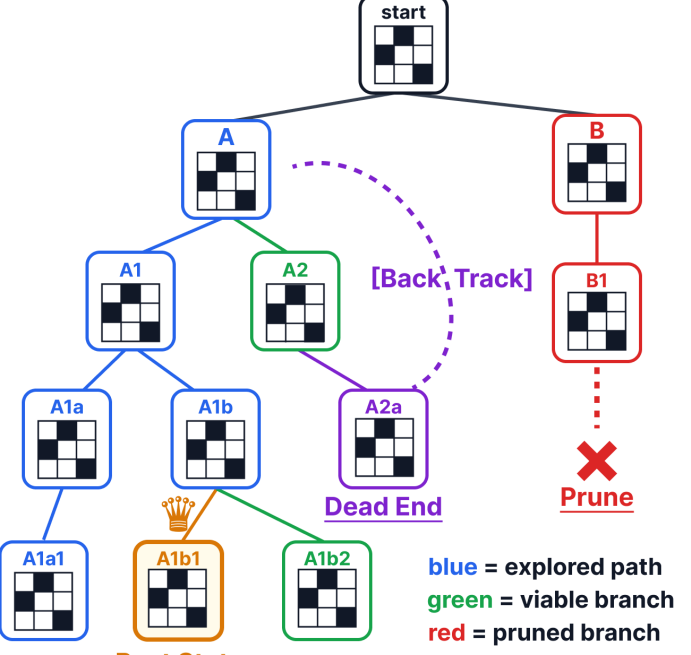
- Problem:** LLMs decode one path at a time. One bad early step ruins the final answer.
- Paper idea:** ToT turns reasoning into a search tree: generate candidate thoughts, evaluate them, and backtrack when needed.
- Our work:**
 - Reproduced ToT on Game of 24, Crosswords, and Creative Writing using modern Gemini models.
 - Tested batched vs. un-batched evaluation on Game of 24 and Crosswords.
 - Developed two remedial strategies: a zero-cost verifier tiebreaker and a half-batching scheme.
 - Extended Creative Writing with post-generation refinement and score-based candidate selection.



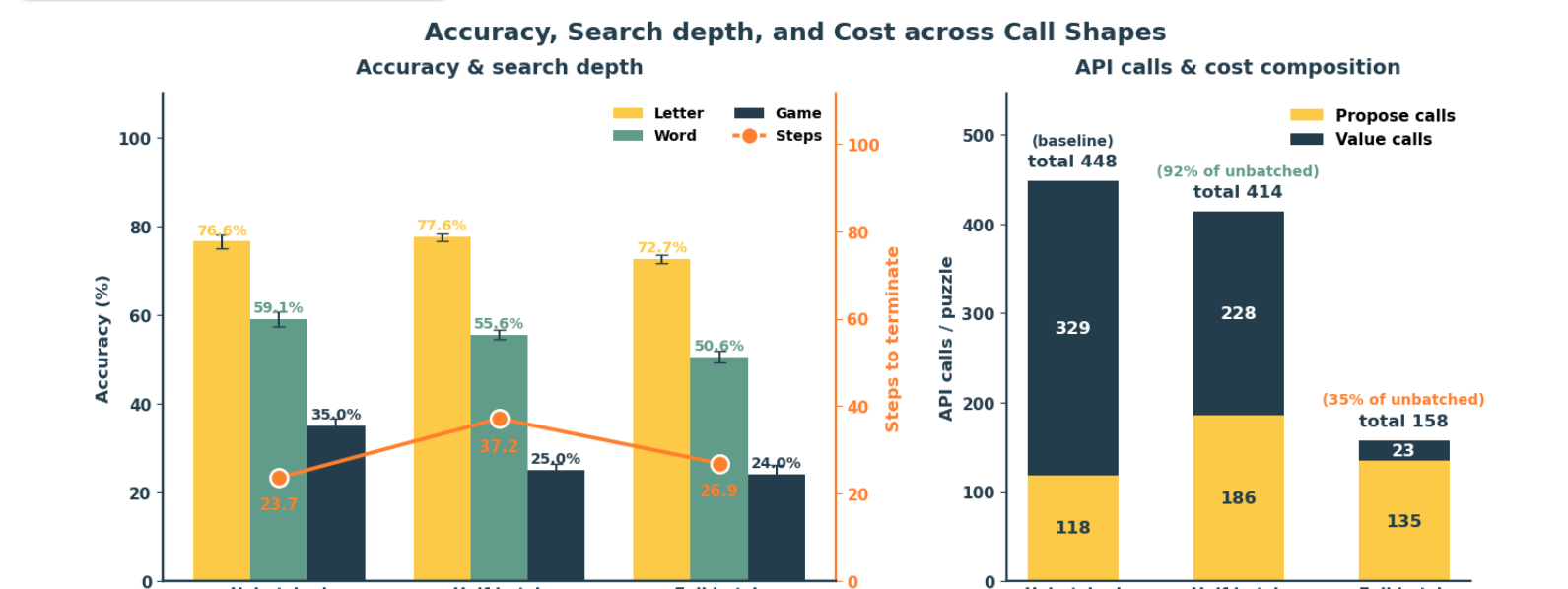
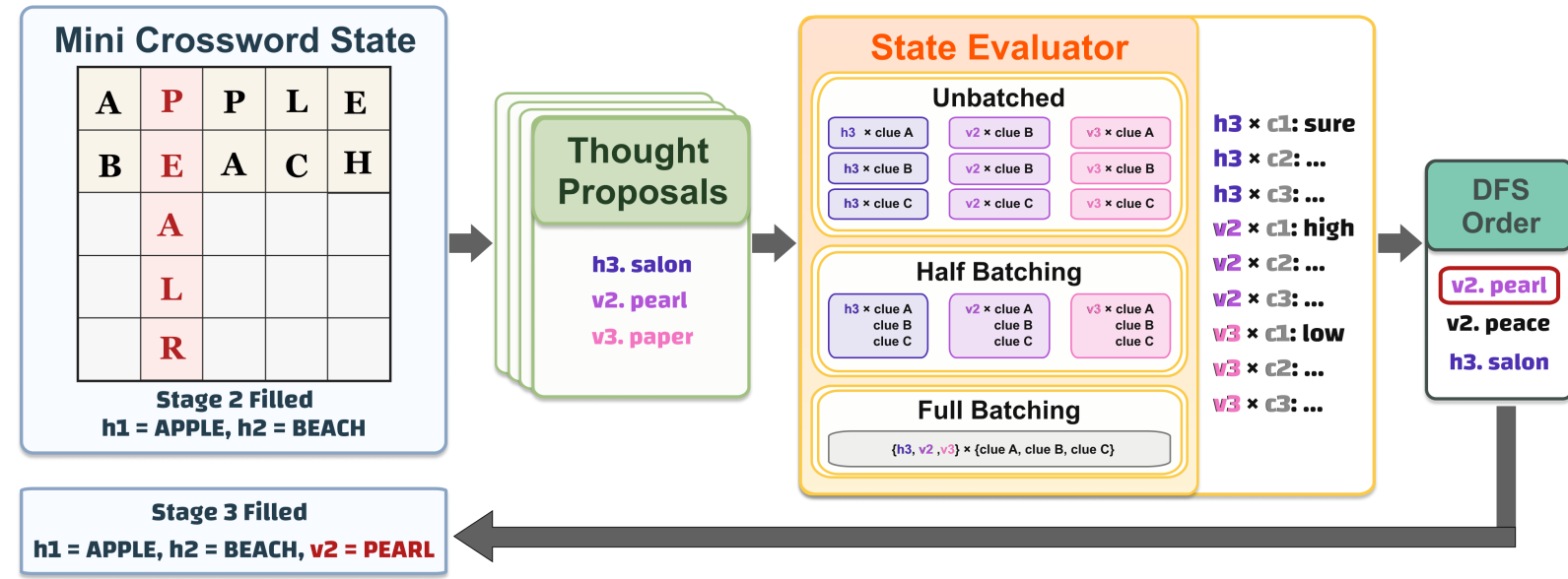
Crosswords: Main Results

Avg. Success Rate (%) of 5 Runs of 20 Puzzle Games	Letter Acc.		Word Acc.		Game Acc.		Avg. Steps (Ours)
	Paper (GPT-4)	Ours (Gemini 3.1)	Paper	Ours	Paper	Ours	
IO	38.7%	45.9%	14.0%	18.5%	0.0%	0.0%	1.0
CoT	40.6%	38.8%	15.6%	15.2%	1.0%	1.0%	1.0
ToT (unbatched)	78.0%	76.6%	60.0%	59.1%	20.0%	35.0%	23.7
→ + best state	82.4%	81.2%	67.5%	64.0%	35.0%	37.1%	19.9
→ - prune	65.4%	69.13%	41.5%	46.5%	5.0%	20.0%	18.6
→ - backtrack	54.6%	37.5%	20.0%	22.2%	5.0%	3.3%	3.5

- Backtracking is the engine.** Removing backtracking makes ToT nearly greedy.
- Pruning earns its keep.** Removing pruning keeps too many weak branches.
- Gem-3.1 outperformed GPT-4 with stronger word-level recall and constraint reasoning, but still has boosts via ToT.
- Improvement from best-state is limited:** Our DFS commits to the right grid when it finds it. Only 14% had a better intermediate state than final

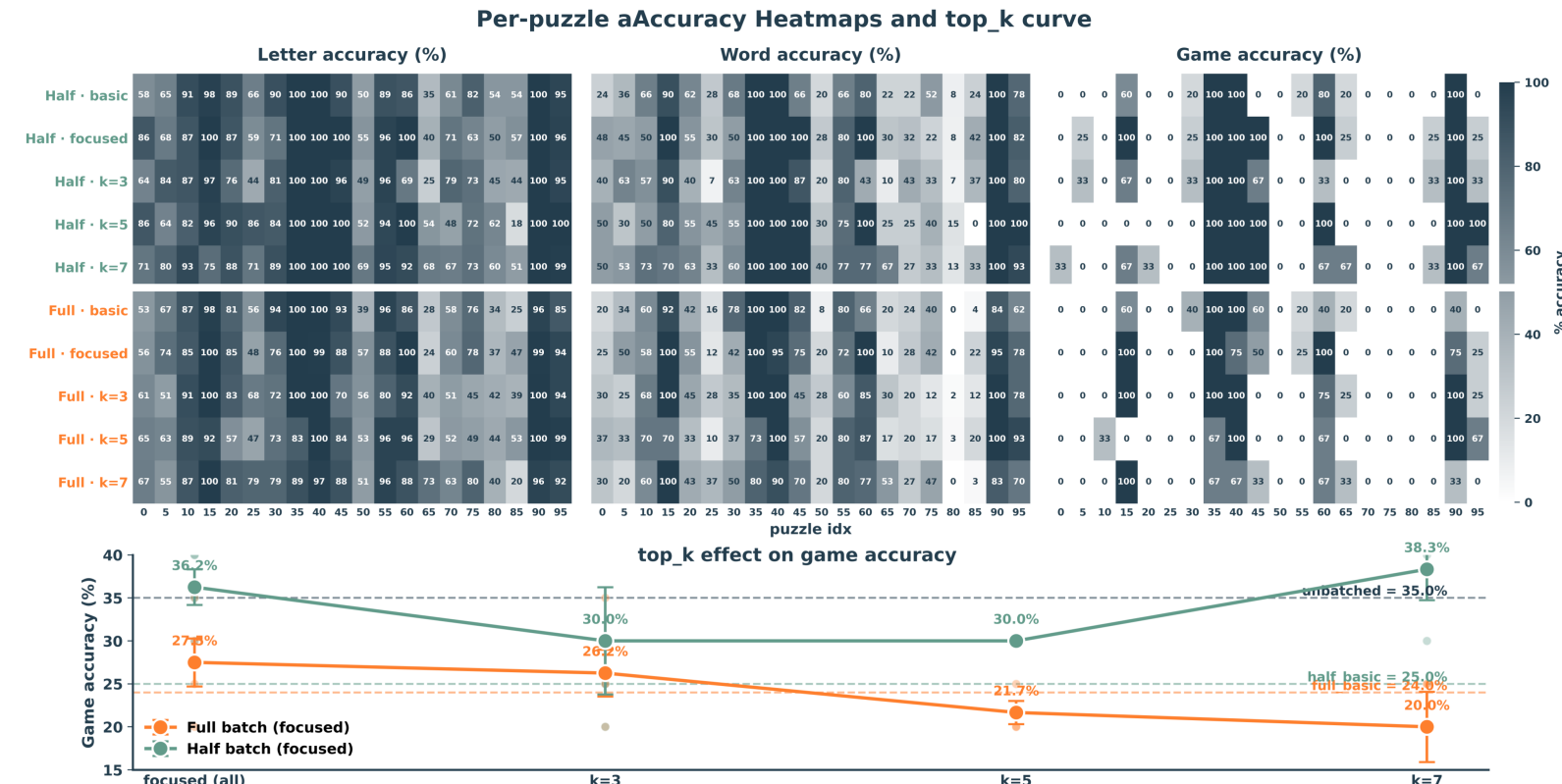


Extension: The Batching Problem

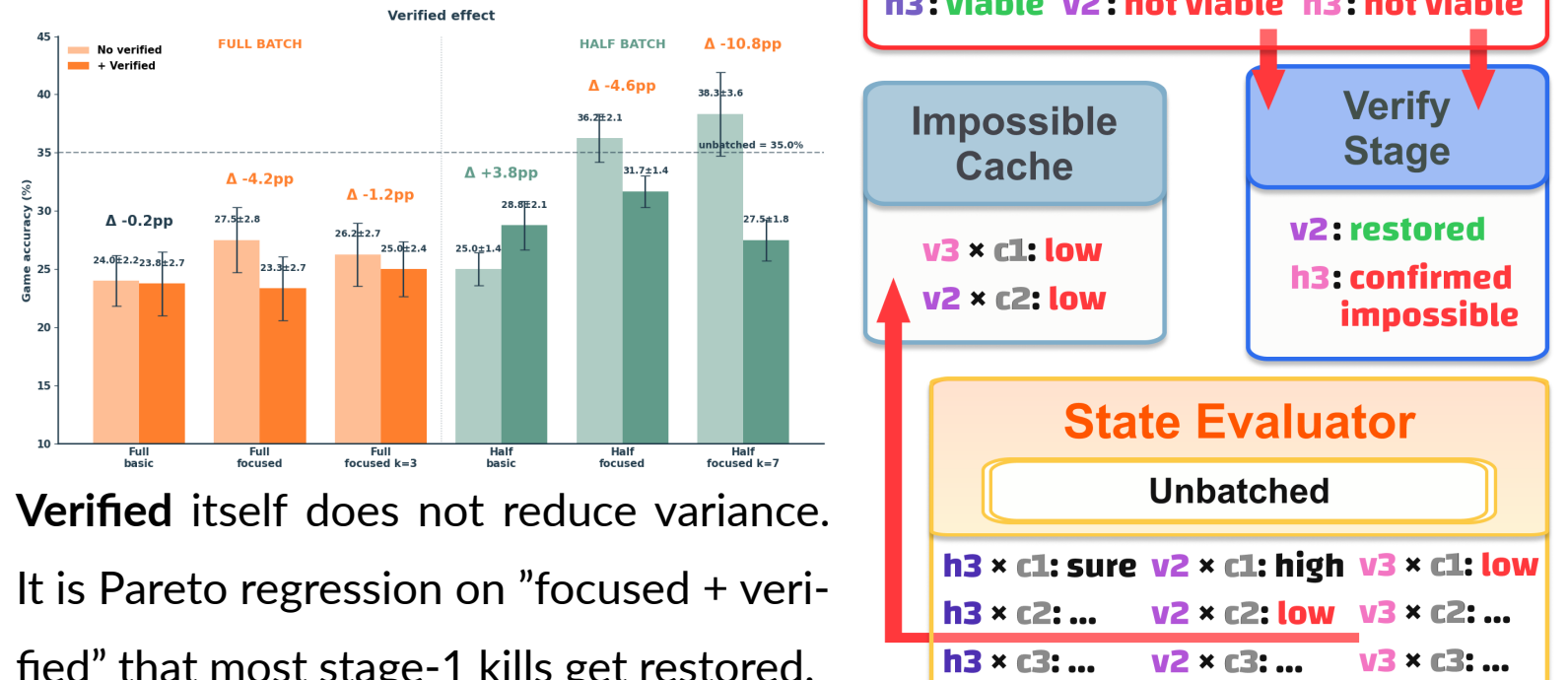


- Full batch:** 65% cheaper than unbatched (158v.s.448 calls). The long list noise drops game accuracy (35.0%→24.0%)
- Half batch:** 11% cheaper than unbatched but loses 9% on game acc. It is **most stable** one with **highest letter accuracy (77.6%)**. It gains local context via larger average search depth (and API calls).

Increase Accuracy for Batched Evaluator



- Full batch:** $top_k = 0$ (all testable clues) (27.5%) and $top_k = 3$ (26.2%) outperformed
 - Half batch:** $top_k = 0$ (36.2%) and $top_k = 7$ (38.3%) are tied
- Focused** dominates on both batch levels. accurate↑ and cheaper



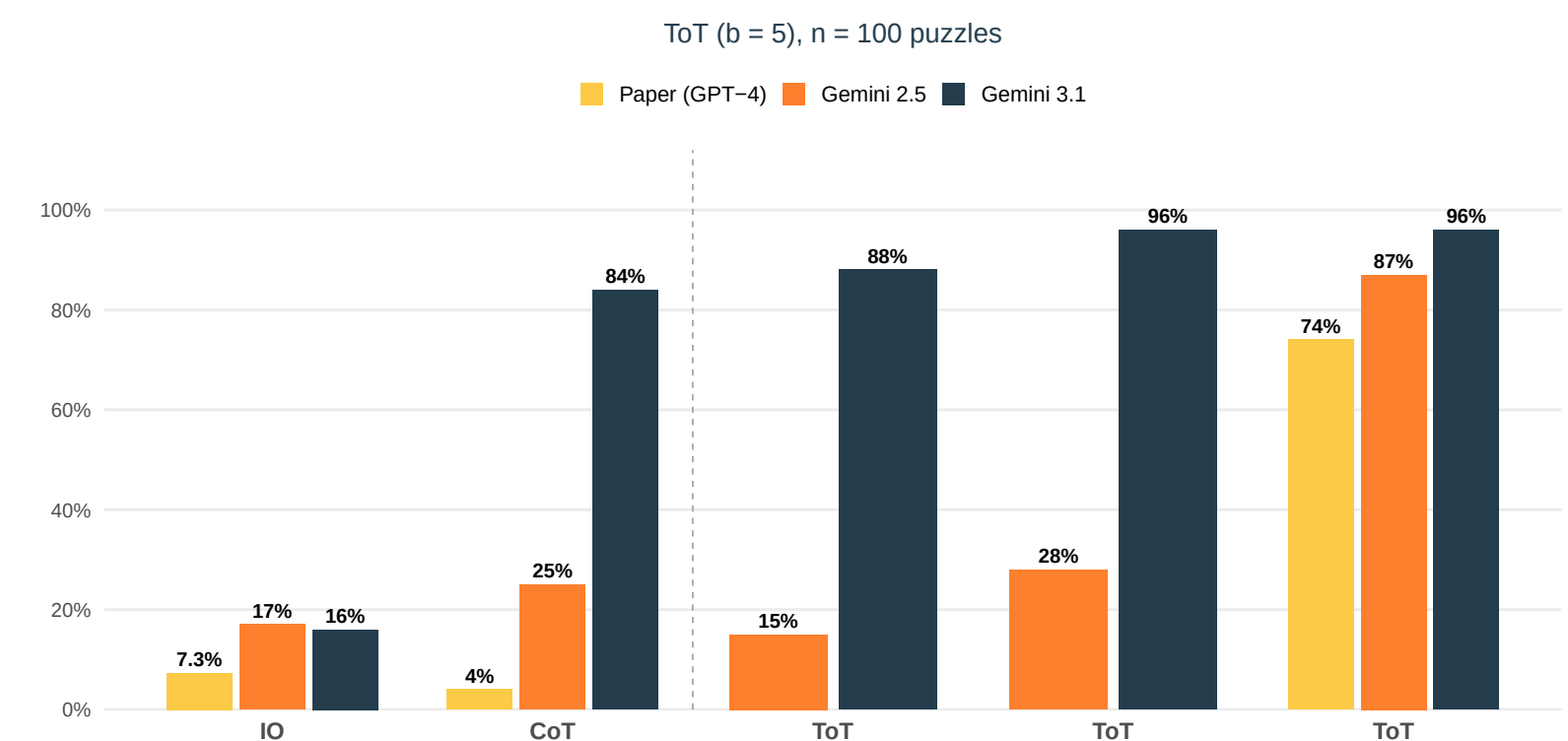
Verified itself does not reduce variance. It is Pareto regression on "focused + verified" that most stage-1 kills get restored.

Decrease Cost for Unbatched Evaluator

Config	Game ± SE	Letter	Word	Calls	Propose	Value	Steps
Unbatched (paper)	35.0 ± 1.8	76.6	59.1	448	118	329	23.7
Unbatched + cache	40.0 ± 4.1	77.1	61.5	347	135	212	27.0

Cache cuts cost without hurting accuracy.

Game of 24: Main Results



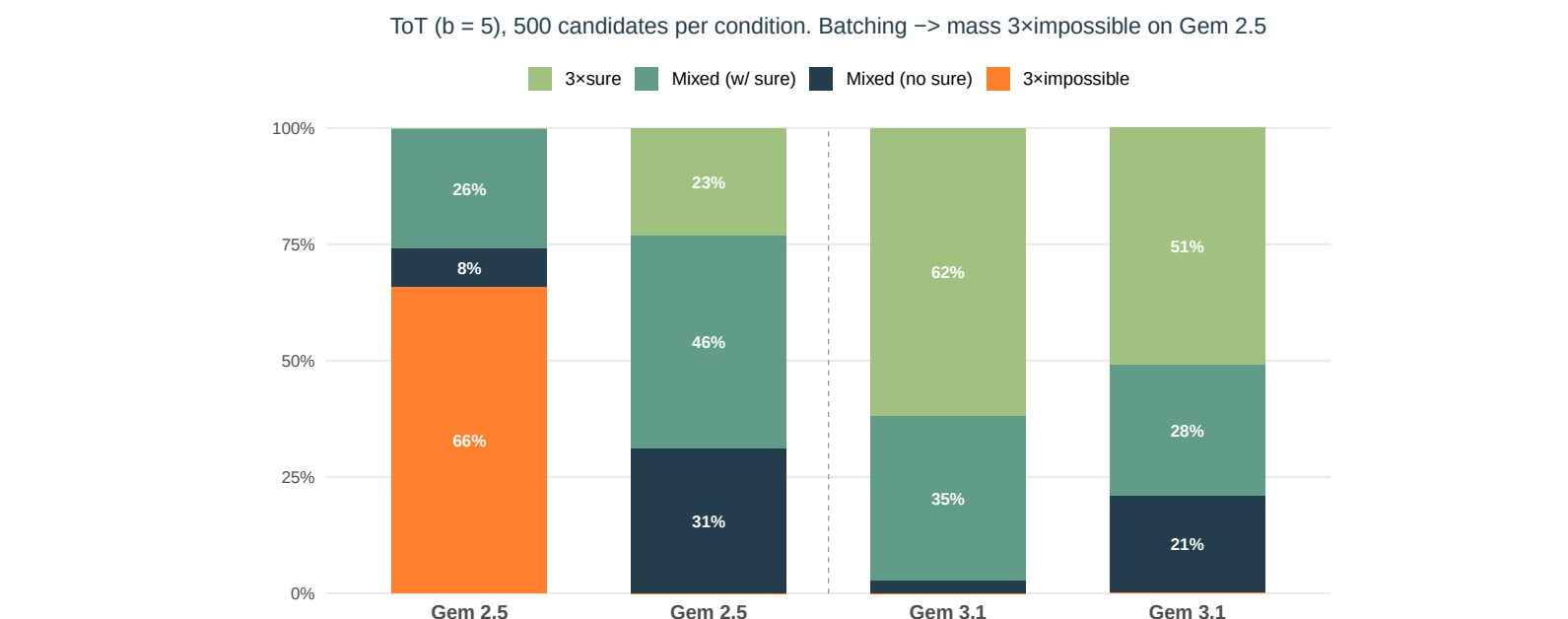
Both Gemini models exceed GPT-4's 74% with un-batched ToT. Batching hurts Gem 2.5 (87%→15%) and Gem 3.1 (96%→88%); the tiebreaker fully recovers Gem 3.1 (88%→96%) and partially recovers Gem 2.5 (15%→28%) at zero extra API cost.

Extension: The Batching Problem

Cost of Un-batching (ToT $b = 5$)

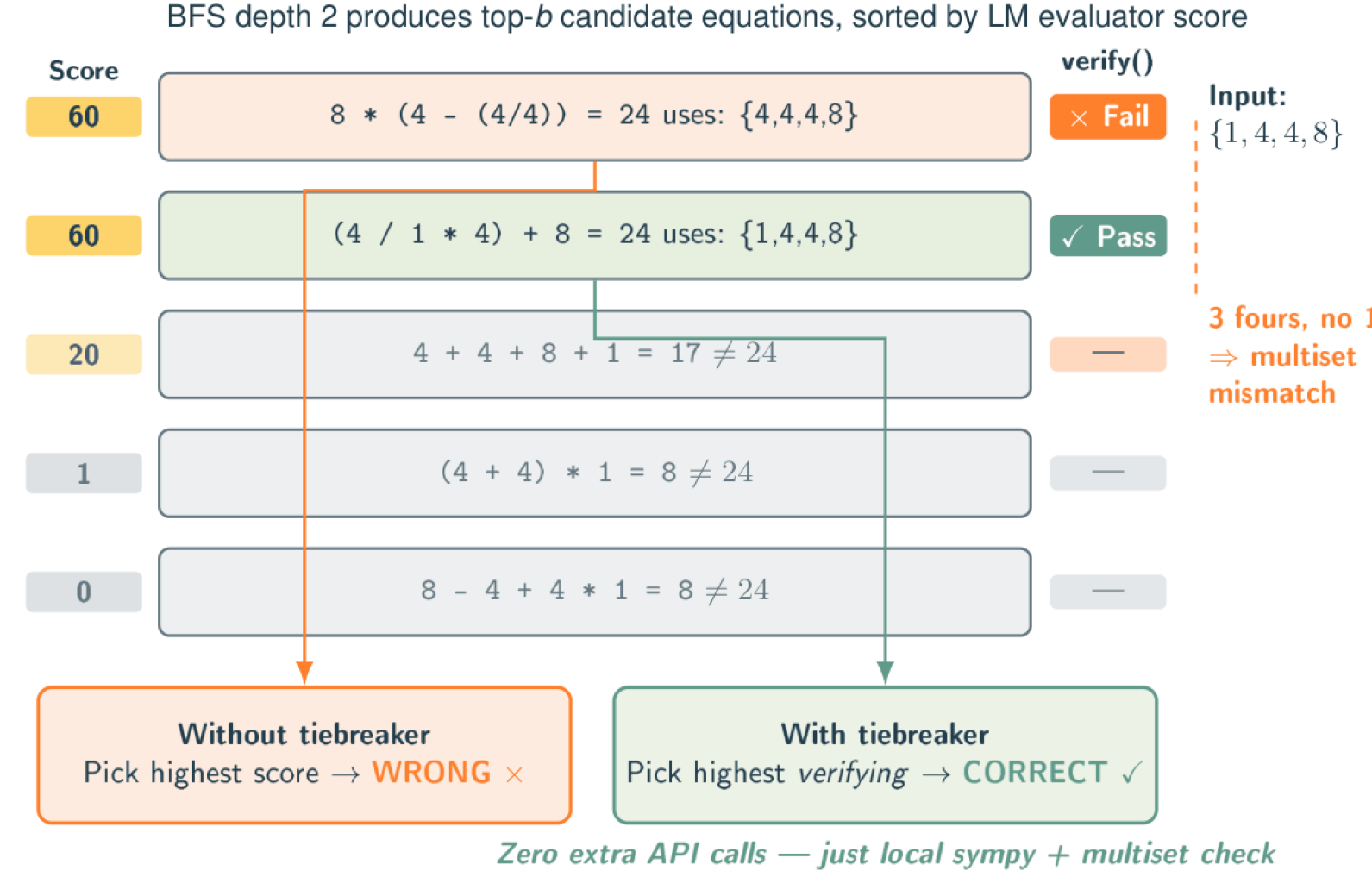
	Cost / 100	Time / puzzle	API calls
Gemini 2.5			
Batched	\$0.32	~8 s	~20
Un-batched	\$0.99	~70 s	~170
Gemini 3.1			
Batched	\$0.77	~12 s	~20
Un-batched	\$3.52	~90 s	~170

Why Batching Hurts



Batched Gem 2.5 votes "impossible" on 66% of depth-0 candidates (vs. 0% un-batched). Gem 3.1 resists: 62% "sure" when batched.

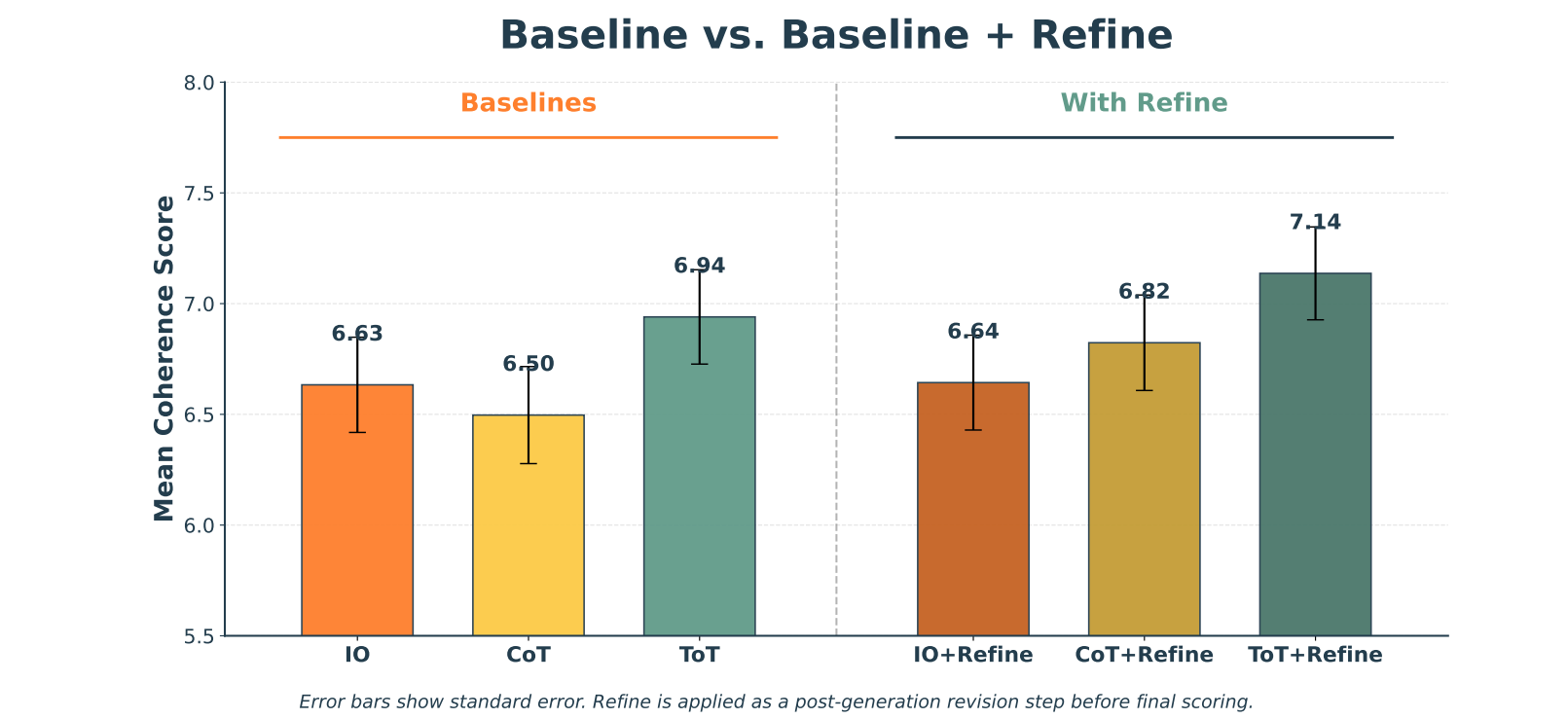
Verifier Tiebreaker



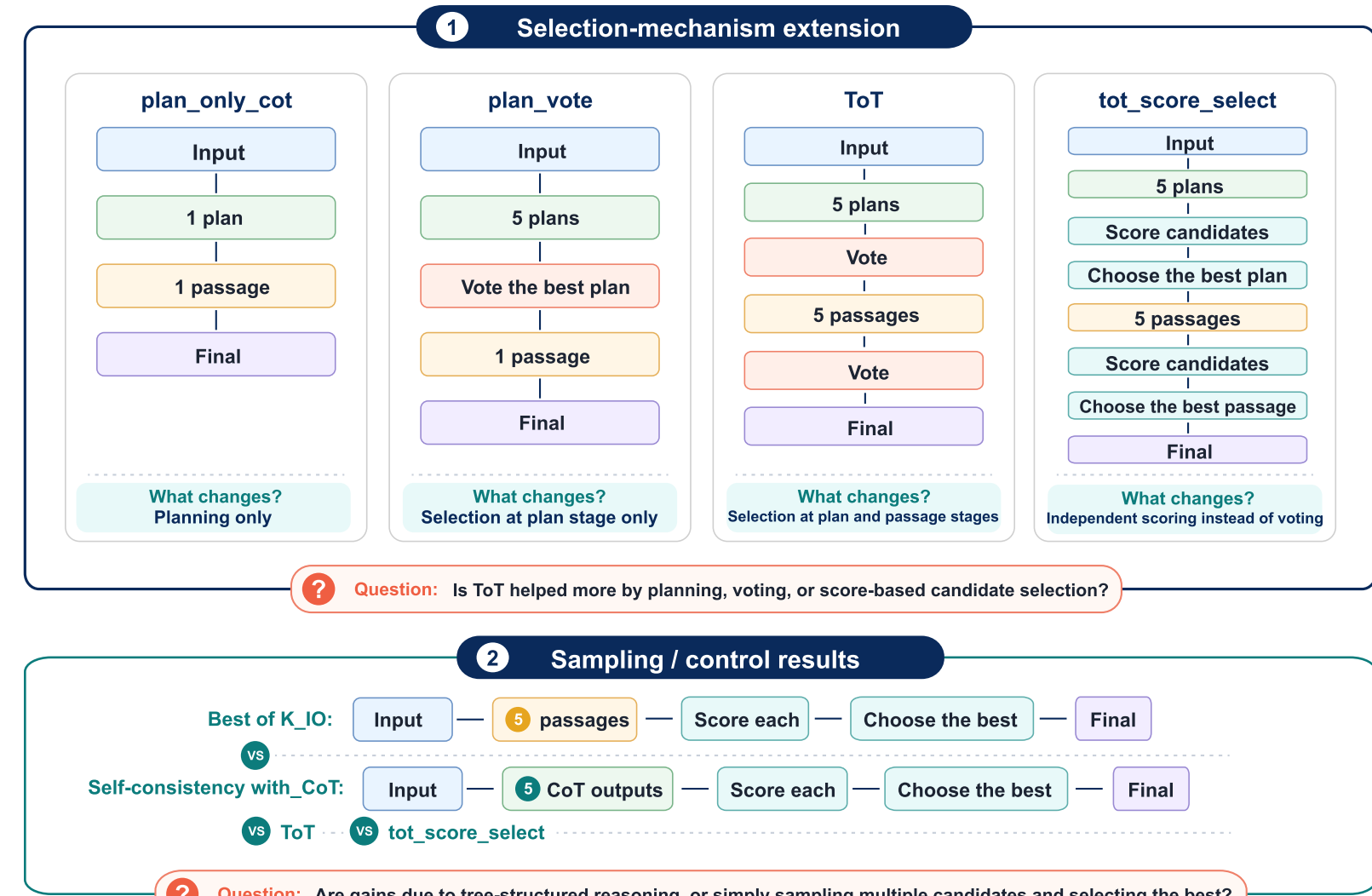
Creative Writing

Main baselines + Refinement

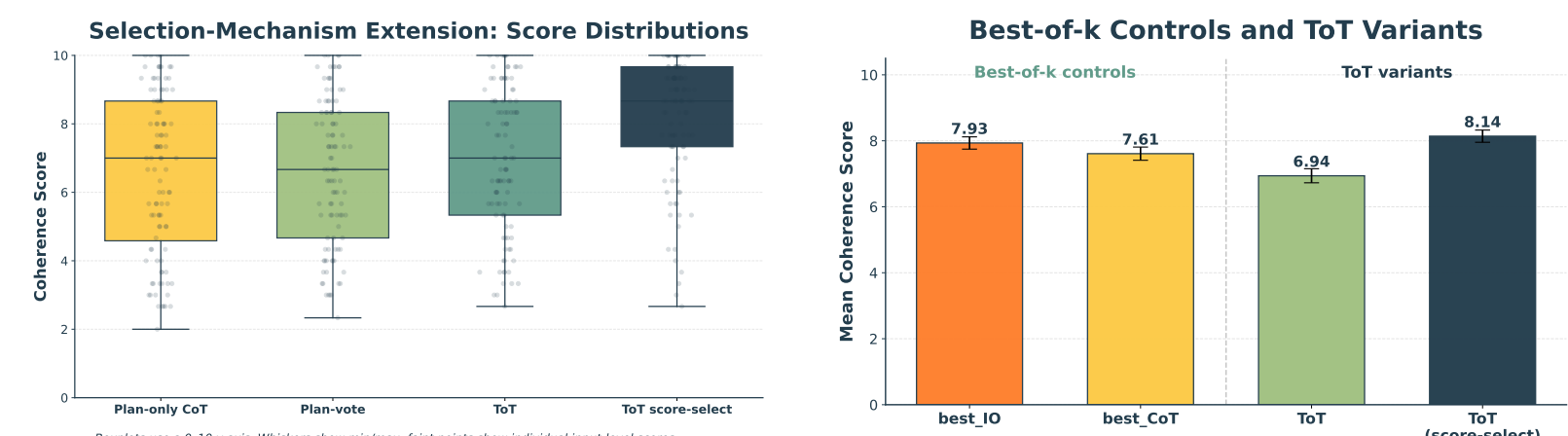
Across IO, CoT, and ToT, compare the baseline with addition of a post-generation revision step.



Extension: Isolate Selection vs Sampling Effects



ToT score-select achieves the highest mean coherence score (8.14), with scores more concentrated near the top range. **Best-of-k IO** and **self-consistency with CoT** also outperform standard ToT, suggesting that sampling multiple candidates is a strong baseline.



The main gain appears to come from candidate scoring/selection rather than standard vote-based ToT.

Conclusions

- ToT's gains replicate** on 2026 models without batching.
- Batching silently breaks ToT** across applicable tasks, i.e. Game of 24 and Crosswords, whether with Gem 2.5 or Gem 3.1. Remedial strategies close the accuracy gap for Gem 3.1.
- For Creative Writing**, ToT benefits less from voting and more from strong candidate selection (score-based selection outperforms standard vote-based ToT).

References

- [1] Shunyu Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS, 2023.
- [2] Zhoujun Cheng, Jungo Kasai, and Tao Yu. *Batch Prompting: Efficient Inference with Large Language Model APIs*. EMNLP Industry Track, 2023.
- [3] Jianzhe Lin et al. *BatchPrompt: Accomplish More with Less*. ICLR, 2024.